

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Database System (BCSE302P)

Project - Review I

Library Management System using MongoDB

TEAM:
Tanishtha Swami (24BDS0245)
Mahati Nishtala (24BDS0188)

Table of Contents

Sr No.	Title	Page no.
1	Problem Statement	3
2	Project Objectives	3
3	Scope and System Boundaries	4
4	Selection and Justification of NoSQL Database	5
5	Data Model Design & ER-to-NoSQL Mapping	8
6	Entity–Relationship Diagram	18
7	Conclusion	17

Problem Statement

Anyone who has tried to borrow a popular textbook during exam week already knows the pain points this project is trying to fix. Most university libraries still run on a patchwork of legacy software, spreadsheets, and manual processes that were never really built for a high-volume, multi-format collection. A few problems keep showing up in practice:

1. **You can't trust the "available" status.** Copy-level status (issued, reserved, under repair, lost) is often updated in an overnight batch job rather than the moment a book is scanned out, so a student can walk to the shelf and find a book already gone even though the system said it was free.
2. **The catalog doesn't fit neatly into rows and columns.** A hardcover textbook, an e-book, an audiobook, and a multi-volume reference set all have genuinely different attributes worth tracking — page count versus runtime versus file format versus DRM license. Forcing all of that into a fixed set of relational columns means either a lot of NULL values or an awkward sprawl of extra tables.
3. **Fines get worked out by hand more often than they should.** Overdue charges are frequently calculated manually or through a disconnected spreadsheet, which leads to disputes and no real audit trail connecting a fine to the loan it came from, the payment made against it, and the receipt handed to the student.
4. **There's no fair way to reserve a book that's checked out.** When a prescribed lab manual is fully issued, students usually end up informally asking a librarian to "hold it for them," which isn't transparent and isn't really first-come-first-served.
5. **Requesting a new title is basically an email and a hope.** There's no formal channel for a student or faculty member to ask the library to acquire a book, and even informal requests rarely get tracked from submission through to whether the book actually showed up.
6. **None of this holds up as the university grows.** As student numbers, campus count, and the digital collection all expand, a single relational server chaining JOINs across books → copies → loans → fines → payments starts to become a real bottleneck, especially during the start-of-semester rush.

Project Objectives

With the previously stated problems in mind, this project is aiming for the following, stated in concrete and testable terms:

1. **Schema Design (O1) :** Design a MongoDB document schema that can natively represent heterogeneous catalog media types without needing schema migrations.

2. **Sub-second Catalog Search (O2) :** Use compound and text indexes so title/author/genre search and availability lookups stay fast even once the catalog grows past 100,000 titles.
3. **Transactional Circulation Integrity (O3) :** Guarantee that issue/return/renewal operations update book_copies.status and books.availableCopies atomically, using MongoDB multi-document transactions, so a single physical copy can never be issued to two people at once.
4. **Automated Fine Auditing (O4) :** Compute overdue fines automatically from the gap between dueDate and returnDate, and keep a fully auditable, append-only payment trail per fine.
5. **Fair Hold Queue Management (O5) :** Maintain a FIFO reservation queue per title so that when a copy is returned, the next eligible member in line is notified automatically.
6. **Procurement Workflow (O6) :** Provide a status-tracked book request pipeline (PENDING → APPROVED/REJECTED → ORDERED → FULFILLED) that's visible to both the requester and the reviewing librarian.
7. **Horizontal Scalability (O7) :** Design the schema and shard-key strategy so the system can grow across multiple campus branches without needing a schema redesign later.

Scope and System Boundaries

To keep this review focused, the system covers five core modules — the same five the problem statement above points to:

In-Scope Modules

Module	Description
Cataloging	Master bibliographic records, physical/digital copy tracking, barcode/accession management, rack location, condition tracking.
Circulation	Issue, return, renewal workflows; active/overdue/returned loan lifecycle; loan history retention.
Fine Auditing	Automatic overdue fine calculation, partial/full payment recording, waiver workflow, audit trail.
Hold Queues	Per-title reservation queue, FIFO ordering, automatic notification trigger when a copy becomes available.
Procurement Requests	Member-submitted new-title requests, librarian review workflow, linkage to the eventual catalog entry once fulfilled.

Selection and Justification of NoSQL Database

The Database Paradigm Landscape

Before settling on MongoDB, it's worth walking through what the realistic alternatives actually were. Four fundamentally different paradigms were on the table:

- **Relational Database (SQL)** — e.g., PostgreSQL/MySQL — normalized tables, foreign keys, JOIN-based query resolution, strict schema.
- **Graph Database** — e.g., Neo4j — nodes and edges, built for deep relationship traversal.
- **Wide-Column / Columnar Store** — e.g., Apache Cassandra — partitioned column families, built for massive linear write throughput.
- **Document Database** — e.g., MongoDB — self-describing BSON/JSON documents, with the option to embed related data.

Comparative Technical Evaluation

Evaluation Dimension	**Relational (PostgreSQL/ MySQL)**	**Graph (Neo4j)**	**Columnar (Cassandra)**	**Document (MongoDB)**
Core data unit	Row in a fixed-column table	Node + labeled edge	Column family / wide row	Self-describing BSON document

Schema flexibility	Rigid; ALTER TABLE needed for new attributes; a poor fit for heterogeneous book formats	Flexible node properties, but relationship-first modeling adds overhead for what's really flat catalog data	Flexible per-row columns, but schema-per-partition design is awkward to evolve	Fully dynamic — each document can carry a different set of fields, which is ideal for hardcover vs. e-book vs. audiobook records
Relationship handling	Excellent, via declarative foreign keys and JOIN	Best-in-class for deep, multi-hop traversals (e.g., "recommend books co-borrowed by similar members")	Weak — denormalization is mandatory, there's no native join	Solid for shallow 1:N/N:M relationships via references, and embedding removes the need for a join on the hot paths
Consistency model (CAP)	Strong consistency, ACID by default (CP-leaning)	Strong consistency (CP)	Tunable, but defaults toward eventual/AP-leaning for high availability	Strong consistency on the primary by default, tunable read/write concerns, and multi-document ACID transactions since v4.0
Read performance for nested/ hierarchical data	Poor — needs multiple JOINs across books, authors, copies, and cost grows with normalization depth	Fine for traversal-shaped queries, but not built for flat catalog reads	Poor — column families aren't designed for hierarchical documents	Strong — a single document read (say, a book with embedded authors and genre) returns fully hierarchical data with zero joins

Write scalability / horizontal scaling	Difficult — vertical scaling or manual, complex sharding	Harder to shard, since graph locality matters	Built from the ground up for linear write scaling across commodity nodes	Native sharding with configurable shard keys, scaling both reads and writes
Query language expressiveness	SQL — mature, declarative, powerful aggregates	Cypher — expressive for pattern-matching traversals	CQL — SQL-like, but intentionally limited (no joins, no arbitrary WHERE on non-indexed columns)	The Aggregation Pipeline — multi-stage analytical queries (grouping, faceting, $lookup when a join is actually needed) directly on JSON-shaped data
Best-fit real-world use case	Financial ledgers, ERP systems needing strict multi-table integrity	Social networks, fraud-ring detection, recommendation graphs	IoT telemetry, time-series, write-heavy logging at large scale	Content catalogs and semi-structured records for applications whose entities keep evolving
Suitability for this LMS	Medium — strong consistency, but schema rigidity works against O1 and JOIN latency works against O2	Low — this system has no deep multi-hop traversal need, so the overhead isn't justified	Low — eventual consistency directly conflicts with O3, where double-issuing a copy isn't an option	High — directly satisfies O1, O2, O3, O4, and O7

Why MongoDB Is the Optimal Choice

MongoDB's document model addresses each of the four angles this criterion is really asking about.

Schema flexibility for a genuinely mixed catalog. Because MongoDB doesn't force a fixed set of columns onto every document, a books collection can hold a hardcover title

with a physicalAttributes.pages field sitting right next to an audiobook document with a runtimeMinutes field and no pages field at all. No migration, no wasted NULL columns — the schema just reflects whatever the item actually is, which is how a real academic catalog behaves. This is the schema flexibility Objective O1 is asking for.

Fast reads, because the data that's used together lives together. The single most frequent operation in this system is "show me a book's details and tell me if it's available." Because a book's authors, genre tags, and other core attributes are embedded rather than split across separate tables, that operation resolves as one document fetch instead of a multi-table JOIN — which is the biggest cost driver in relational catalog systems once volume picks up. That's what makes Objective O2's sub-second search realistic.

A real aggregation engine for fine auditing. The Aggregation Pipeline ($match, $group, $sum, $facet) can answer something like "total outstanding fines this month, grouped by department" in a single query. In SQL that's a multi-table join plus a group-by; in Cassandra, it's essentially not expressible without pre-building a materialized view. This is what Objective O4's auditing requirement depends on.

Room to grow across campuses. book_copies can be sharded on branch, so that each campus's circulation traffic — which is almost entirely writes, since every issue and return touches this collection — spreads across separate shard primaries instead of piling onto one server. A single-instance relational deployment would need significant re-architecture to get anywhere close, which is exactly what Objective O7 is trying to avoid having to do later.

Data Model Design & ER-to-NoSQL Mapping

Relational design leans on normalization — get to third normal form and stop there. MongoDB doesn't really work that way. The question isn't "is this normalized," it's "how is this data actually going to be read." For every relationship in the model, the same test was applied: if a piece of data is almost always read alongside its parent, changes at roughly the same rate as its parent, and won't grow without bound, it gets embedded. If it's queried independently, written to constantly, or could grow indefinitely, it gets referenced by ObjectId instead. That one distinction drives every embedding decision described below.

ER Entity → Collection Mapping

Conceptual ER Entity	MongoDB Collection	Cardinality from Related Entities
Member / Librarian (generalized as "User")	users	1 user → N loans, N fines, N requests
Book (bibliographic work)	books	1 book → N physical copies; 1 book → N queued holds (embedded)
Physical/Digital Copy	book_copies	N copies → 1 book (FK); 1 copy → N historical loans
Borrow Transaction (Loan)	borrow_transactions	1 member → N transactions; 1 copy → N transactions over time
Fine (with nested Payments)	fines_and_payments	1 transaction → 0..N fines; 1 fine → N payment attempts (embedded)
Procurement Request	book_requests	1 member → N requests; 1 request → 0..1 fulfilled book

One thing worth flagging here: the hold queue doesn't get its own collection. A reservation queue only ever makes sense in the context of one specific book, and it's read at the exact moment someone checks whether a copy is available — so it's embedded directly inside the books document as reservationQueue[] rather than living in a seventh collection. The reasoning behind that call is laid out below.

Detailed Collection Schemas, Keys, and Sample Data

Collection: users

Purpose: One collection covers every kind of system user — students, faculty, librarians, and admins — with a role field acting as a discriminator that determines which extra fields actually apply to that document.

Key Type	Field	References

Primary Key	`_id` (ObjectId)	—
—	(no outbound FK; this is a root entity)	—

Embedding decisions:

- Address, authentication details, membership status, and role-specific fields are all embedded directly in the user document. There's no real scenario in this application where a user's profile is needed without also needing their contact info or membership status, so splitting those into separate collections would just mean extra round trips for no real benefit.

Sample Document:


```
{
  "_id": { "$oid": "66c1a2b3c4d5e6f7a8b9c0d1" },
  "role": "STUDENT",
  "fullName": "Ananya Sharma",
  "email": "ananya.sharma2022@vitstudent.ac.in",
  "phone": "+91-9876543210",
  "address": {
    "street": "Hostel Block D, Room 214",
    "city": "Vellore",
    "state": "Tamil Nadu",
    "zipCode": "632014",
    "country": "India"
  },
  "authentication": {
    "passwordHash":
"$2b$12$K9xV7mZ8pQ1wR4tY6uOeXeJhLd3nS0aF2bC5dE7gH9i",
    "mfaEnabled": false,
    "lastLoginAt": { "$date": "2026-08-20T09:15:00Z" },
    "failedLoginAttempts": 0
  },
  "membership": {
    "membershipId": "VIT-LIB-2024-08841",
    "joinDate": { "$date": "2024-08-01T00:00:00Z" },
    "expiryDate": { "$date": "2028-07-31T00:00:00Z" },
    "status": "ACTIVE",
    "maxBooksAllowed": 4,
    "currentBooksIssued": 2,
    "totalOutstandingFine": { "$numberDecimal": "15.00" }
  },
  "roleDetails": {
    "rollNumber": "22BCE1487",
```

```
    "department": "School of Computer Science Engineering",
    "program": "B.Tech CSE",
    "semester": 5
  },
  "accountStatus": "ACTIVE",
  "createdAt": { "$date": "2024-08-01T10:00:00Z" },
  "updatedAt": { "$date": "2026-08-20T09:15:00Z" }
}
```


Collection: books

Purpose: The master catalog — one document per distinct title/edition, kept completely separate from however many physical copies of it happen to exist.

Key Type	Field	References
Primary Key	`_id` (ObjectId)	—
Embedded reference	reservationQueue[].memberId	→ users._id

Embedding decisions:

- Authors, publisher info, genre tags, and physical attributes are embedded because they're small, they barely ever change, and they show up on practically every catalog search result — fetching them from a separate collection on every search would be overhead for no real gain.
- The hold queue is the more interesting call. It's embedded as reservationQueue[] instead of being broken out into its own holds collection because it only ever makes sense in the context of one book, it's read at the same moment as that book's availability, and in practice it's never going to run past a few dozen entries before library policy caps it anyway. Each entry just stores a lightweight memberId reference back to users, so the array itself stays small even though it's pointing at full member records elsewhere.
- totalCopies and availableCopies are denormalized counters, kept in sync with the book_copies collection inside the same transaction as any issue or return, so the catalog can answer "how many copies are free right now" without running a separate aggregation on every search.

Sample Document:


```
{
  "_id": { "$oid": "66c1b1c2d3e4f5a6b7c8d9e0" },
  "isbn13": "9780262046305",
  "isbn10": "0262046302",
```

```
  "title": "Introduction to Algorithms",
  "subtitle": "Fourth Edition",
  "authors": [
    { "name": "Thomas H. Cormen", "role": "AUTHOR" },
    { "name": "Charles E. Leiserson", "role": "CO_AUTHOR" },
    { "name": "Ronald L. Rivest", "role": "CO_AUTHOR" },
    { "name": "Clifford Stein", "role": "CO_AUTHOR" }
  ],
  "publisher": { "name": "MIT Press", "country": "USA" },
  "edition": "4th",
  "language": "English",
  "publicationYear": 2022,
  "genre": ["Computer Science", "Algorithms", "Reference"],
  "physicalAttributes": { "pages": 1312, "format": "HARDCOVER" },
  "totalCopies": 6,
  "availableCopies": 0,
  "reservationQueue": [
    {
      "memberId": { "$oid": "66c1a2b3c4d5e6f7a8b9c0d1" },
      "queuedOn": { "$date": "2026-08-15T14:22:00Z" },
      "queuePosition": 1,
      "status": "WAITING"
    },
    {
      "memberId": { "$oid": "66c1a2b3c4d5e6f7a8b9c0f2" },
      "queuedOn": { "$date": "2026-08-18T11:05:00Z" },
      "queuePosition": 2,
      "status": "WAITING"
    }
  ],
  "createdAt": { "$date": "2022-07-01T00:00:00Z" },
  "updatedAt": { "$date": "2026-08-20T16:40:00Z" }
}
```

Collection: book_copies

Purpose: Tracks every individually barcoded item — physical or digital-licensed — separately from the catalog record it belongs to.

Key Type	Field	References
Primary Key	`_id` (ObjectId)	—
Foreign Key	bookId	→ books._id

Referencing decision (not embedding): This one stays a separate, referenced collection rather than living inside books, for basically the opposite reasons that made the hold queue a good candidate for embedding. A copy's status field changes on almost every circulation event, and a popular title can accumulate hundreds of copies across branches over the years. Embedding that inside books would mean rewriting the entire parent document on every single issue or return, and would risk running into real document-size and write-contention problems as the collection grows — exactly the kind of high-write, unbounded relationship referencing is meant to handle.

Sample Document:


```
{
  "_id": { "$oid": "66c1c1d2e3f4a5b6c7d8e9f0" },
  "bookId": { "$oid": "66c1b1c2d3e4f5a6b7c8d9e0" },
  "barcode": "VITLIB-CSE-004821",
  "accessionNumber": "ACC/2022/3311",
  "branch": "Main Campus - Central Library",
  "rackLocation": { "block": "B-Wing", "shelf": "CS-14", "row": "R3" },
  "condition": "GOOD",
  "status": "ISSUED",
  "acquisitionDate": { "$date": "2022-07-10T00:00:00Z" },
  "price": { "$numberDecimal": "2450.00" },
  "lastMaintenanceDate": null,
  "createdAt": { "$date": "2022-07-10T00:00:00Z" },
  "updatedAt": { "$date": "2026-08-05T10:12:00Z" }
}
```


Collection: borrow_transactions

Purpose: The circulation ledger itself — one document per issue event, tracking it from the moment a book goes out to the moment it (hopefully) comes back.

Key Type	Field	References
Primary Key	`_id` (ObjectId)	—
Foreign Key	memberId	→ users._id
Foreign Key	copyId	→ book_copies._id
Foreign Key	issuedBy	→ `users._id` (librarian)
Foreign Key	returnedTo	→ `users._id` (librarian), nullable

Embedding decisions:

- bookSnapshot deliberately duplicates the ISBN and title as they were at the moment of issue — the Extended Reference Pattern. If a librarian later corrects a typo in the book's title, or the title is withdrawn from the catalog entirely, old loan records shouldn't silently change or break. They should read exactly as they did on the day the book was actually issued.
- renewalHistory is embedded as a small array because library policy caps renewals at two or three per loan, so it can't grow out of control, and it's always read together with the rest of the loan record anyway.

Sample Document:


```
{
  "_id": { "$oid": "66c1d1e2f3a4b5c6d7e8f9a0" },
  "memberId": { "$oid": "66c1a2b3c4d5e6f7a8b9c0d1" },
  "copyId": { "$oid": "66c1c1d2e3f4a5b6c7d8e9f0" },
  "issuedBy": { "$oid": "66c1a2b3c4d5e6f7a8b9c0e5" },
  "returnedTo": null,
  "bookSnapshot": {
    "bookId": { "$oid": "66c1b1c2d3e4f5a6b7c8d9e0" },
    "isbn13": "9780262046305",
    "title": "Introduction to Algorithms"
  },
  "issueDate": { "$date": "2026-08-05T10:12:00Z" },
  "dueDate": { "$date": "2026-08-19T23:59:59Z" },
  "returnDate": null,
  "renewalCount": 1,
  "maxRenewalsAllowed": 2,
  "renewalHistory": [
    {
      "renewedOn": { "$date": "2026-08-18T09:00:00Z" },
      "previousDueDate": { "$date": "2026-08-19T23:59:59Z" },
      "newDueDate": { "$date": "2026-09-02T23:59:59Z" }
    }
  ],
  "status": "ACTIVE",
  "remarks": null,
  "createdAt": { "$date": "2026-08-05T10:12:00Z" },
  "updatedAt": { "$date": "2026-08-18T09:00:00Z" }
}
```


Collection: fines_and_payments

Purpose: One document per fine, with its own payment history embedded right inside it — covering fine tracking and payment auditing in a single place instead of two.

Key Type	Field	References
Primary Key	`_id` (ObjectId)	—
Foreign Key	`loanId`	→ `borrow_transactions._id`
Foreign Key	`memberId`	→ `users._id`
Foreign Key	`waivedBy`	→ `users._id` (librarian/admin), nullable

Embedding decision: Payment attempts live inside paymentHistory[] rather than in a standalone payments collection. A fine realistically gets settled in one, maybe two or three installments — never an unbounded number — and whenever anyone actually needs to audit how a fine was paid, they need the fine and its payments together, not as two separate lookups. Keeping them in one document also means the fine's current status and its full payment trail update atomically in a single write, rather than needing two collections to stay in sync with each other.

Sample Document:


```
{
  "_id": { "$oid": "66c1e1f2a3b4c5d6e7f8a9b0" },
  "loanId": { "$oid": "66c1d1e2f3a4b5c6d7e8f9a0" },
  "memberId": { "$oid": "66c1a2b3c4d5e6f7a8b9c0d1" },
  "waivedBy": null,
  "reason": "OVERDUE",
  "amount": { "$numberDecimal": "20.00" },
  "paidAmount": { "$numberDecimal": "5.00" },
  "balanceAmount": { "$numberDecimal": "15.00" },
  "status": "PARTIALLY_PAID",
  "calculatedOn": { "$date": "2026-08-20T00:00:00Z" },
  "waivedReason": null,
  "paymentHistory": [
    {
      "paymentId": "PMT-2026-000482",
      "amount": { "$numberDecimal": "5.00" },
      "gateway": "UPI",
      "transactionId": "UPI2026082011223344",
      "receiptNumber": "RCPT-88213",
      "status": "SUCCESS",
      "paymentDate": { "$date": "2026-08-20T11:20:00Z" },
      "gatewayResponse": {
        "upiRefId": "112233445566",
        "payerVpa": "ananya22@okhdfcbank"
```

```
    }
   }
  ],
  "createdAt": { "$date": "2026-08-20T00:00:00Z" },
  "updatedAt": { "$date": "2026-08-20T11:20:00Z" }
}
```


Collection: book_requests

Purpose: Tracks a procurement request from the moment a member submits it, through review, and — eventually — fulfillment.

Key Type	Field	References
Primary Key	`_id` (ObjectId)	—
Foreign Key	requestedBy	→ users._id
Foreign Key	reviewedBy	→ `users._id` (librarian/ admin), nullable
Foreign Key	fulfilledBookId	→ `books._id`, nullable (populated only once the title is actually procured and

Referencing decision: fulfilledBookId is a nullable reference rather than an embedded copy of the eventual catalog entry, because the request and the book that eventually results from it have completely separate lifecycles. There's no reason a request document should need to be touched every time a librarian edits that book's catalog metadata months or years later.

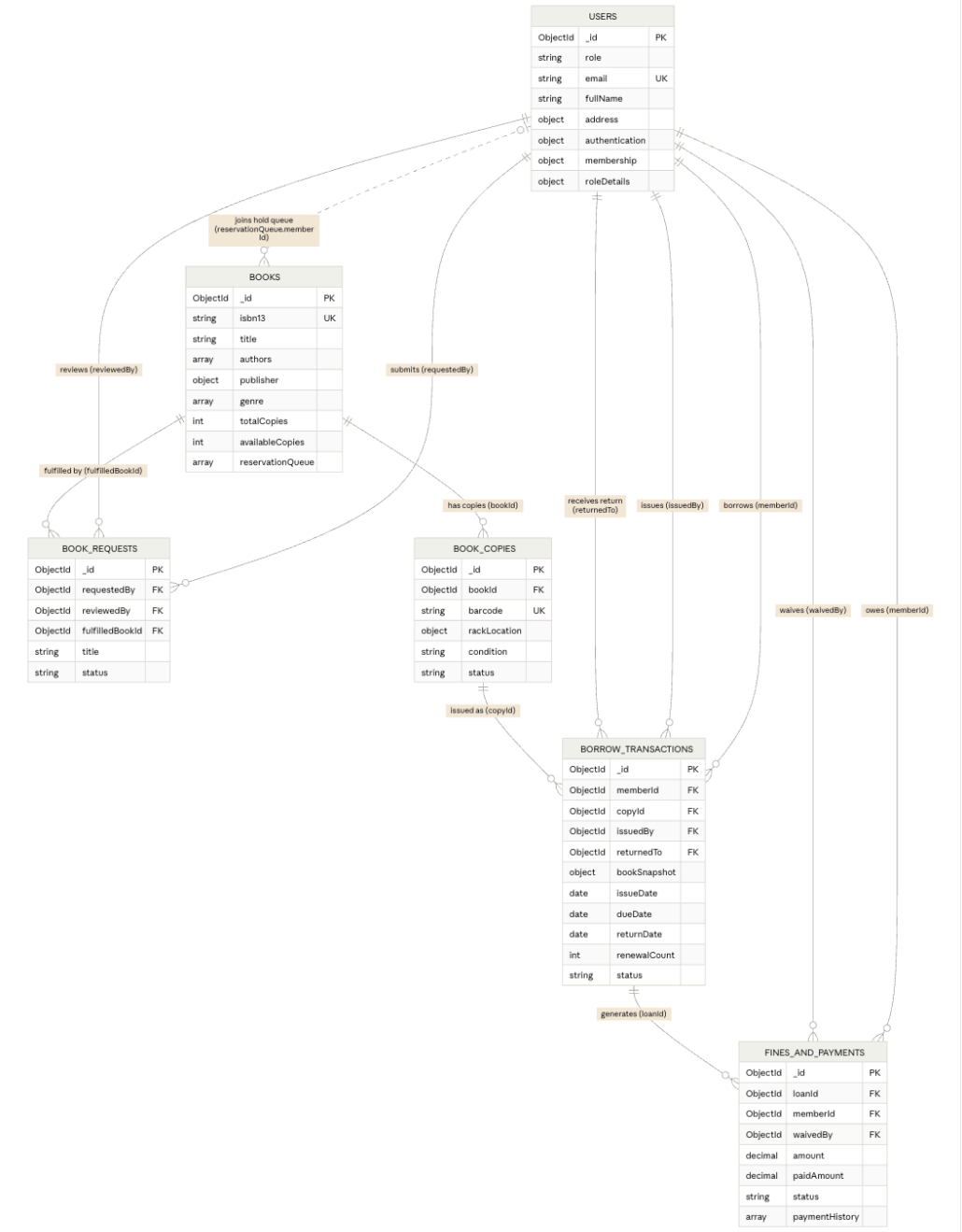
Sample Document:


```
{
  "_id": { "$oid": "66c1f1a2b3c4d5e6f7a8b9c0" },
  "requestedBy": { "$oid": "66c1a2b3c4d5e6f7a8b9c0d1" },
  "reviewedBy": { "$oid": "66c1a2b3c4d5e6f7a8b9c0e5" },
  "fulfilledBookId": null,
  "title": "Designing Data-Intensive Applications",
  "authorName": "Martin Kleppmann",
  "isbn": "9781449373320",
  "publisher": "O'Reilly Media",
  "edition": "1st",
  "justification": "Required as supplementary reading for BCSE302P Database Systems Lab; not currently available in the catalog.",
```

```
  "priority": "HIGH",
  "status": "APPROVED",
  "reviewComments": "Approved for procurement; 3 copies to be ordered for Central Library.",
  "requestDate": { "$date": "2026-08-10T12:00:00Z" },
  "decisionDate": { "$date": "2026-08-14T15:30:00Z" }
}
```

Entity–Relationship Diagram

The diagram below pulls together all six collections and shows exactly how they reference one another — solid connections are real ObjectId references; the one dotted line represents the embedded hold-queue relationship discussed in 3.2 and 3.3.2, which is a reference in spirit but lives inside an array rather than a separate collection.



Primary Key / Foreign Key Reference Table

Collection	Primary Key	Foreign Key Field	Points To
users	_id	—	Root entity — nothing else in this collection points outward.
books	_id	reservationQueue[].memberId	users._id
book_copies	_id	bookId	books._id
borrow_transactions	_id	memberId	users._id
borrow_transactions	_id	copyId	book_copies._id
borrow_transactions	_id	issuedBy	users._id
borrow_transactions	_id	returnedTo	users._id
fines_and_payments	_id	loanId	borrow_transactions._id
fines_and_payments	_id	memberId	users._id
fines_and_payments	_id	waivedBy	users._id
book_requests	_id	requestedBy	users._id
book_requests	_id	reviewedBy	users._id
book_requests	_id	fulfilledBookId	books._id

Conclusion

By moving away from rigid SQL joins to a practical document model, this architecture fixes the core headaches of university library systems—from lagging availability statuses to fitting awkward media formats into fixed tables. Embedding small, co-located data like hold queues and payment histories keeps common catalog searches fast, while referencing high-frequency entities like individual book copies and transactions prevents document bloat and write contention.

This hybrid approach directly satisfies objectives O1 through O7: it handles diverse media types natively, prevents double-issuing through atomic transactions, and keeps fine audits transparent. With the core entities, ER-to-NoSQL mappings, and shard keys established, this layout gives us a solid, well-structured base to build out compound indexes, aggregation pipelines, and circulation logic in Review 2.
